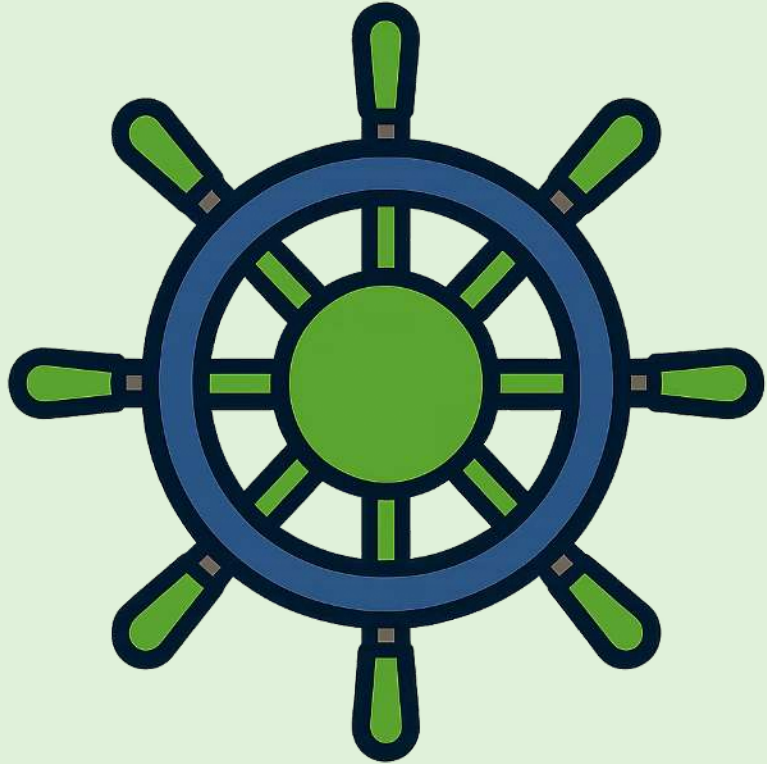




Microservices

Kubernetes, Skalierung & Deployment

Unterrichtseinheit 4/4

Agenda

01

Rückblick UE3

Docker Compose & Multi-Container Apps

03

Einführung Kubernetes

Grundkonzepte & Architektur

05

CI/CD & Deployment

Automatisierter Build- und Release-Prozess

02

Grenzen von Docker Compose

Was fehlt bei größeren Systemen?

04

Skalierung & Load Balancing

Traffic verteilen & Ausfälle abfangen

06

Praxis & Diskussion

Übungen, Transfer & Reflexion

Lernziele

Nach dieser Einheit kannst du folgende Konzepte sicher einordnen und erklären:

Grundkonzepte

Kubernetes-Begriffe wie Pod, Node, Cluster und Deployment erklären

Orchestrierung

Verstehen, wie Container automatisch verwaltet und verteilt werden

Skalierung

Horizontale und vertikale Skalierungsansätze beschreiben

Load Balancing

Traffic-Verteilung auf mehrere Pods einordnen und erklären

Deployment

CI/CD-Pipelines und Deployment-Strategien verstehen

Rückblick: Unterrichtseinheit 3

Docker Compose

Mehrere Services in einer YAML-Datei definieren und gemeinsam starten

Multi-Container Apps

Frontend, Backend und Datenbank als verbundene Container betreiben

Netzwerke & Volumes

Container kommunizieren über interne Netzwerke, Daten werden in Volumes persistiert

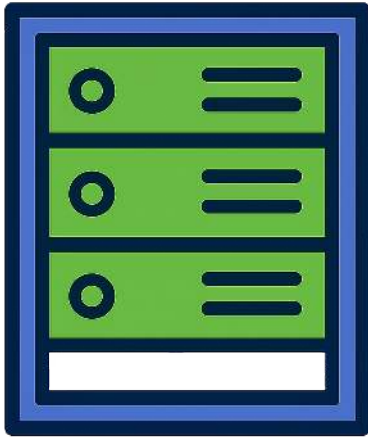
Debugging

Logs analysieren, Container inspizieren und Fehler systematisch eingrenzen

🔍 **Diskussionsfrage:** Was passiert, wenn unsere Anwendung sehr groß wird und Tausende von Anfragen gleichzeitig bearbeiten muss?



Problemstellung: Wachstum braucht mehr



Je größer eine Anwendung wird, desto komplexer werden die Anforderungen an den Betrieb. Einfache Lösungen wie Docker Compose stoßen schnell an ihre Grenzen.

→ Automatische Skalierung

Bei Lastspitzen müssen neue Instanzen spontan starten

→ Verteilung auf mehrere Server

Keine Abhängigkeit von einem einzigen Host

→ Self-Healing

Abgestürzte Container sollen automatisch neu gestartet werden

→ Hochverfügbarkeit

Das System muss rund um die Uhr erreichbar bleiben

Grenzen von Docker Compose

Einzelner Host

Docker Compose läuft in der Regel auf **einem einzigen Rechner** – fällt dieser aus, ist die gesamte Anwendung down.

Keine automatische Skalierung

Services müssen manuell skaliert werden – kein automatisches Reagieren auf Lastspitzen.

Kein Self-Healing

Crasht ein Container, bleibt er down – es gibt keine automatische Wiederherstellung.

Kein Cluster-Management

Verteilung auf mehrere Maschinen und Knoten ist nicht vorgesehen.

Fazit

Docker Compose ist ideal für die lokale Entwicklung und kleine Setups – aber **nicht für produktive, hochverfügbare Systeme** geeignet.

Für den Produktionsbetrieb braucht es ein Orchestrierungssystem wie Kubernetes.



Einführung: Kubernetes

Kubernetes (kurz: K8s) ist ein Open-Source-System zur **Orchestrierung von Containern** – entwickelt von Google, heute ein CNCF-Projekt.



Deployment

Container-Anwendungen
automatisch starten und verwalten



Skalierung

Instanzen dynamisch hoch- und
runterskalieren



Verwaltung

Ressourcen, Netzwerke und Konfigurationen zentral steuern

Die Grundidee von Kubernetes



Kubernetes arbeitet nach dem **Desired-State-Prinzip**: Du beschreibst, wie der Soll-Zustand aussehen soll – Kubernetes sorgt kontinuierlich dafür, dass dieser Zustand eingehalten wird.

i Du sagst Kubernetes **nicht**, wie es etwas tun soll – du sagst ihm, **was** du möchtest. Den Rest erledigt das System automatisch.

Praktisches Beispiel

Statt manuell Container zu starten, definierst du: „*Es sollen immer 3 Instanzen meiner App laufen.*“ Kubernetes startet sie, überwacht sie und startet sie bei Ausfall neu – ohne manuellen Eingriff.

Wichtige Begriffe



Cluster

Die Gesamtheit aller Maschinen (Nodes), die Kubernetes verwaltet. Ein Cluster besteht aus einem Control Plane und mehreren Worker Nodes.



Pod

Die kleinste deploybare Einheit in Kubernetes. Ein Pod enthält einen oder mehrere Container, die gemeinsam laufen.



Node

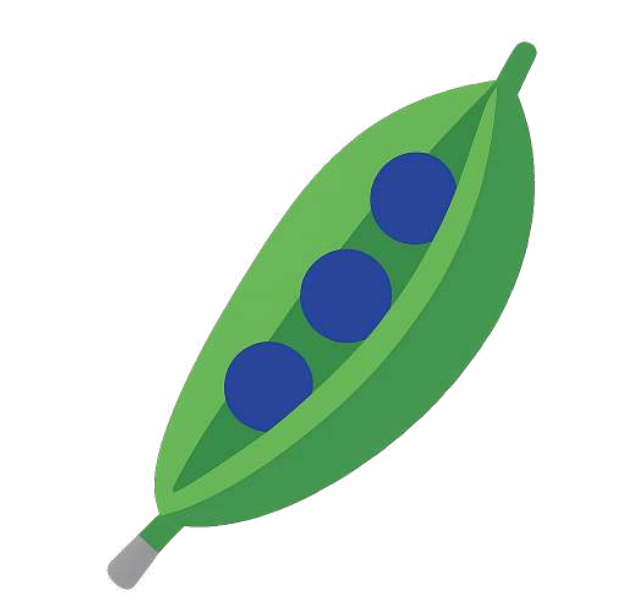
Ein einzelner Server (physisch oder virtuell) im Cluster. Hier laufen die eigentlichen Container-Workloads.



Deployment

Eine Konfiguration, die beschreibt, wie viele Pod-Replikas laufen sollen und welches Container-Image verwendet wird.

Der Pod



KLEINSTE EINHEIT

Was ist ein Pod?

Ein Pod ist die **kleinste deploybare Einheit** in Kubernetes. Er kapselt einen oder mehrere eng zusammengehörende Container, die sich denselben Netzwerkraum und denselben Speicher teilen.

Enthält Container

Meist ein Container pro Pod – bei Bedarf auch mehrere (z.B. Sidecar-Pattern)

Läuft auf einem Node

Kubernetes entscheidet, auf welchem Node ein Pod platziert wird (Scheduling)

Flüchtig

Pods sind temporär – sie können jederzeit neu erstellt werden

Das Deployment

Was beschreibt ein Deployment?

Ein Deployment ist eine **deklarative Konfiguration**, die Kubernetes mitteilt, wie der gewünschte Zustand der Anwendung aussehen soll.

Merksatz: Das Deployment ist der Bauplan – Kubernetes baut und überwacht das Ergebnis automatisch anhand dieses Plans.

Kubernetes vergleicht ständig den *tatsächlichen Zustand* mit dem im Deployment definierten *Soll-Zustand* und korrigiert Abweichungen automatisch.

Anzahl der Pods (Replicas)

Wie viele Instanzen der Anwendung gleichzeitig laufen sollen

Container-Image

Welches Docker-Image für die Pods verwendet werden soll

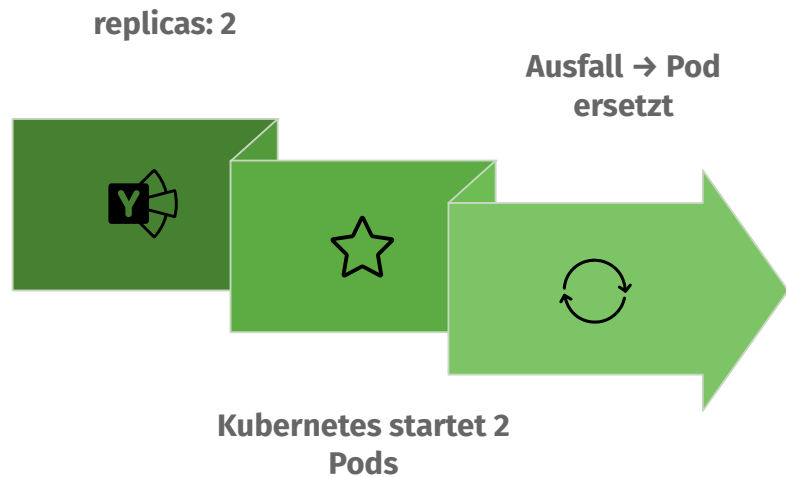
Beispiel: Deployment YAML

Eine einfache Deployment-Definition in Kubernetes sieht so aus:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
      - name: webapp
        image: my-webapp:latest
```

❏ Das Feld `replicas: 2` sorgt dafür, dass Kubernetes immer genau 2 Pods dieser App am Laufen hält.

Erklärung: Replicas



Was bedeutet replicas?

Der Wert replicas gibt an, wie viele **identische Pod-Instanzen** parallel laufen sollen.

Mehrere Instanzen

Kubernetes startet automatisch die gewünschte Anzahl Pods auf verfügbaren Nodes

Selbständige Überwachung

Fällt ein Pod aus, erstellt Kubernetes sofort einen Ersatz – ganz ohne manuellen Eingriff

Erstes Deployment anwenden

Ziel: Ein einfaches Kubernetes Deployment anwenden und laufende Pods anzeigen.

Arbeitsordner: uebung-11-erstes-deployment

Aufgabe

1. Öffne das Manifest `deployment.yaml`.
2. Wende es mit `kubectl apply -f deployment.yaml` an.
3. Prüfe mit `kubectl get deployments` und `kubectl get pods`, was erstellt wurde.
4. Erkläre, warum mehr als ein Pod laufen kann.

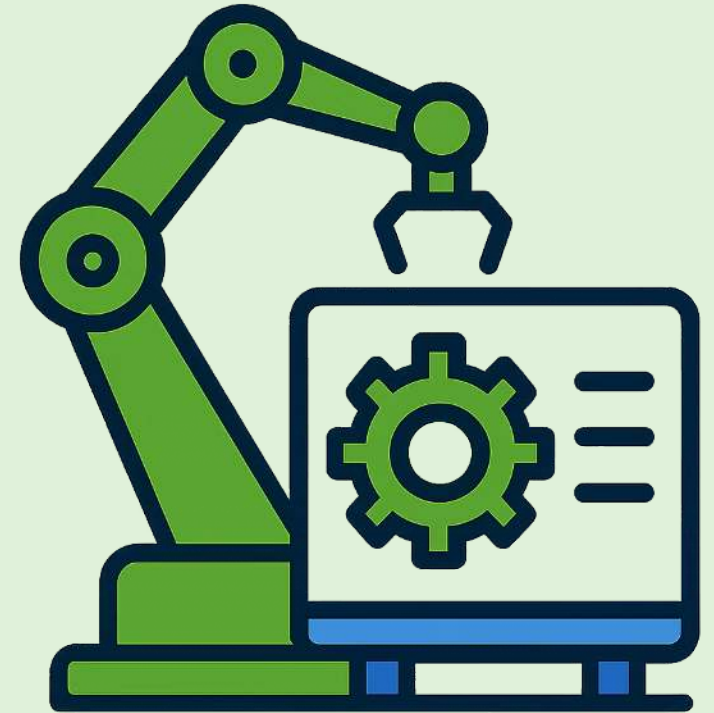
Hilfreiche Befehle

```
kubectl apply -f deployment.yaml  
kubectl get deployments  
kubectl get pods
```

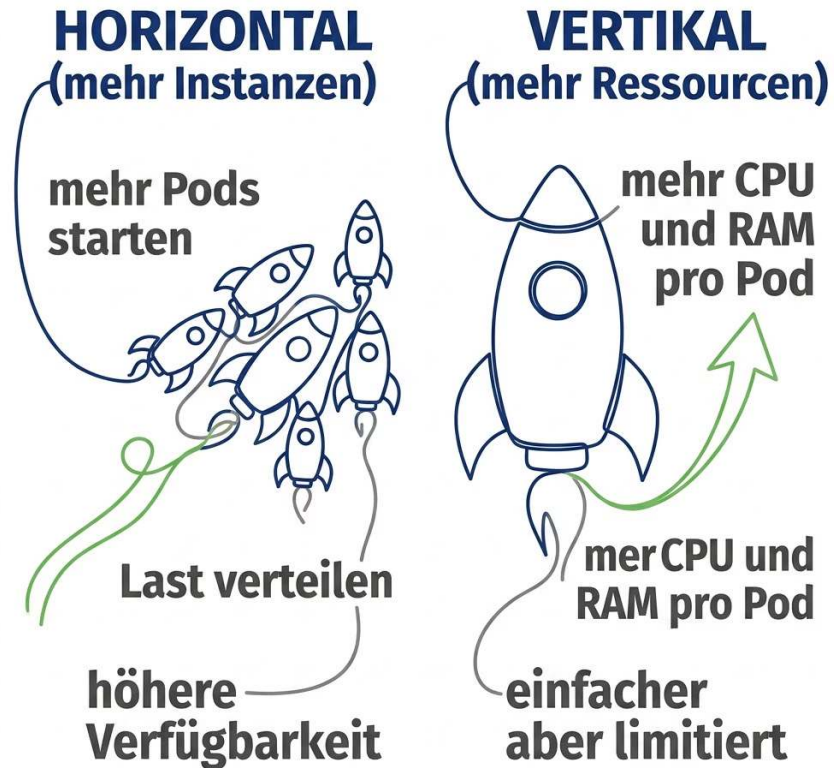
Self-Healing

Kubernetes erkennt Fehler – und behebt sie automatisch.

Sobald ein Pod abstürzt oder nicht mehr antwortet, erkennt Kubernetes diesen Zustand und startet den Pod automatisch neu – ohne manuellen Eingriff. Das System strebt immer den definierten Soll-Zustand an.



Skalierung in Kubernetes



Welche Strategie wählen?

In der Praxis wird meist **horizontale Skalierung** bevorzugt – sie ist flexibler, resilienter und lässt sich in Kubernetes sehr einfach umsetzen.

- i Kubernetes unterstützt auch **automatische horizontale Skalierung** (HPA – Horizontal Pod Autoscaler), die bei Lastspitzen selbstständig neue Pods startet.

Deployment skalieren

Ziel: Die Replikzahl eines Deployments erhöhen und die Änderung beobachten.

Arbeitsordner: uebung-12-deployment-skalieren

Aufgabe

1. Wende das Deployment aus dem Ordner an.
2. Prüfe die aktuelle Replikzahl.
3. Skaliere das Deployment auf 3 Pods.
4. Prüfe danach erneut die laufenden Pods.

Hilfreiche Befehle

```
kubectl apply -f deployment.yaml  
kubectl get deployments  
kubectl scale deployment webapp --replicas=3  
kubectl get pods
```

Services in Kubernetes



⚠ **Problem:** Pods sind flüchtig – sie werden neu gestartet, erhalten neue IP-Adressen und können nicht direkt adressiert werden.

Die Lösung: Der Service

Ein Kubernetes-Service stellt eine **stabile Netzwerkadresse** vor einer Gruppe von Pods bereit. Egal wie oft Pods neu gestartet werden – der Service bleibt erreichbar.

Stabile Adresse

Feste IP/DNS, unabhängig von Pod-Lebenszyklen

Zugriff von außen

Externer Traffic wird über den Service zu Pods geroutet

Load Balancing

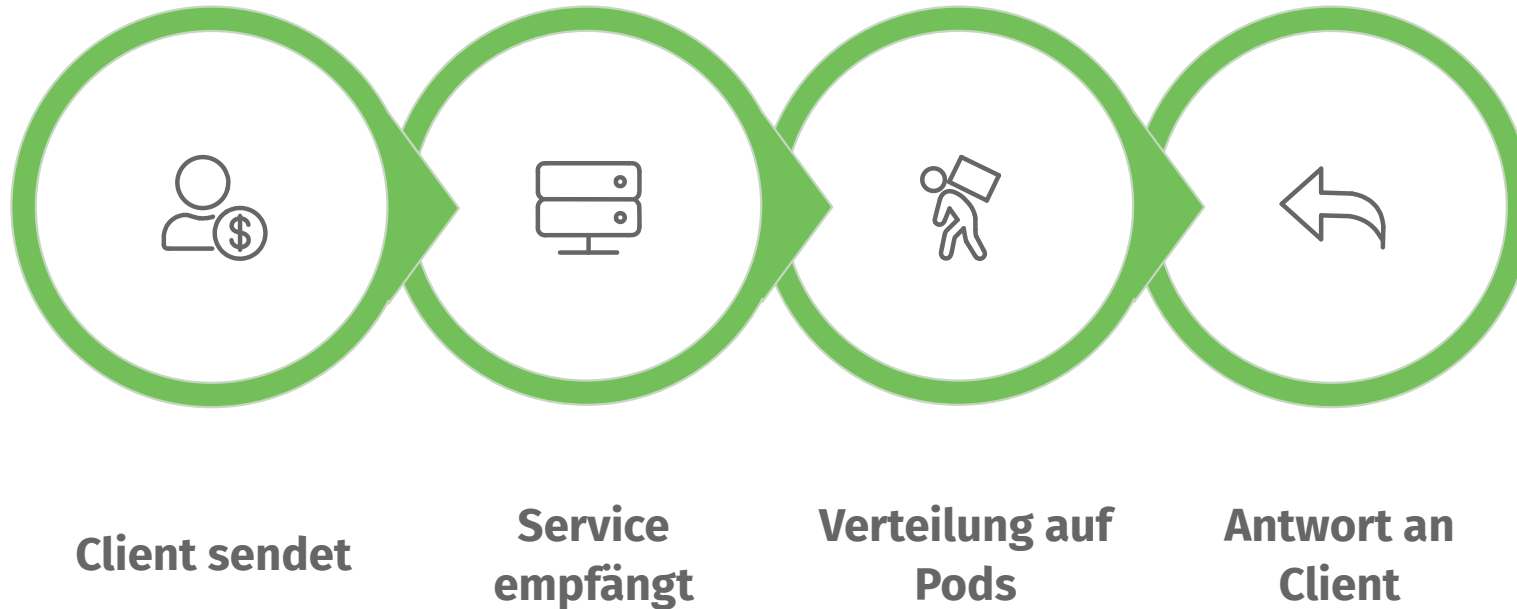


Der Kubernetes Service fungiert gleichzeitig als Load Balancer: Er verteilt eingehende Anfragen gleichmäßig auf alle verfügbaren Pods.

Vorteile des Load Balancing

- **Bessere Performance**
Anfragen werden auf mehrere Pods verteilt – keine einzelne Instanz wird überlastet
- **Ausfallsicherheit**
Fällt ein Pod aus, übernehmen die restlichen Pods den Traffic automatisch
- **Transparenz**
Clients merken nichts von der Verteilung – sie sprechen immer mit dem gleichen Service-Endpunkt

Load Balancing: Beispielablauf



Der Client kommuniziert ausschließlich mit dem Service – welcher Pod die Anfrage letztlich bearbeitet, ist für den Client vollständig transparent.

Service vor ein Deployment setzen

Ziel: Verstehen, wie ein Service Pods über Labels findet und eine stabile Adresse bereitstellt.

Arbeitsordner: uebung-13-service-vor-deployment

Aufgabe

1. Wende `deployment.yaml` und `service.yaml` an.
2. Prüfe mit `kubectl get pods` und `kubectl get svc`, was erstellt wurde.
3. Vergleiche die Labels im Deployment mit dem Selector im Service.
4. Erkläre, warum der Service trotz flüchtiger Pods stabil bleibt.

Hilfreiche Befehle

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods
kubectl get svc
kubectl port-forward svc/webapp-service 8080:80
```

NGINX als Load Balancer



NGINX im Kubernetes-Kontext

NGINX ist ein weit verbreiteter Webserver, der als **Reverse Proxy und Load Balancer** eingesetzt wird – auch in Kubernetes als Ingress-Controller.

Reverse Proxy

NGINX nimmt Anfragen entgegen und leitet sie an interne Services weiter – der Client kennt die Backend-Struktur nicht

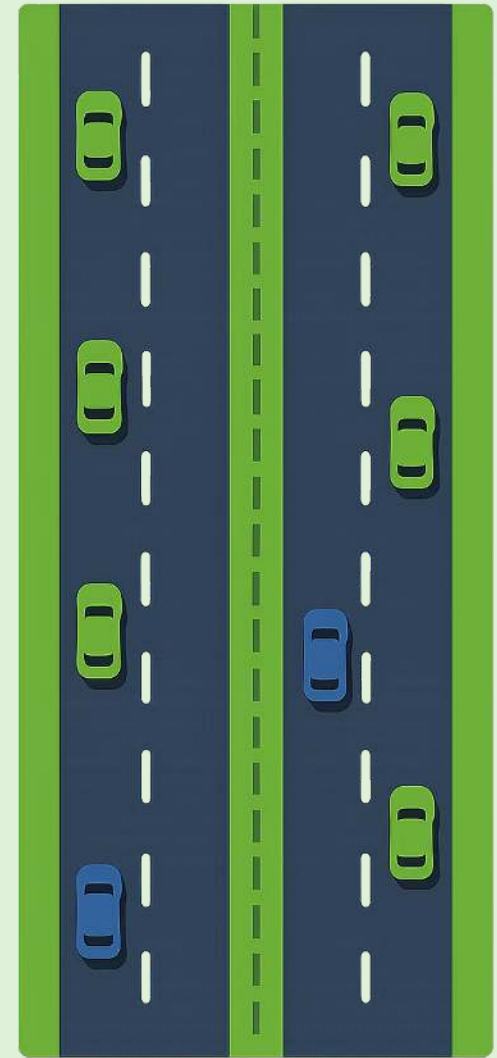
Einfache Konfiguration

Routing-Regeln, SSL-Terminierung und Traffic-Verteilung lassen sich deklarativ konfigurieren

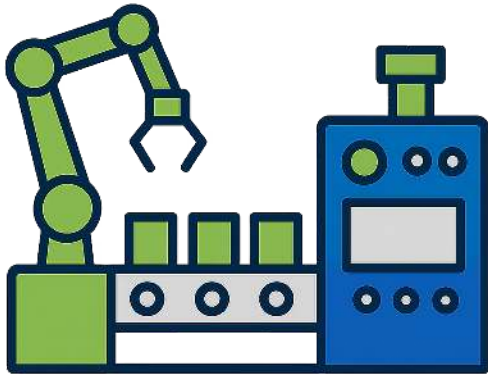
Verbindung zu Microservices

Mehr Services = mehr Traffic = Load Balancing ist unverzichtbar.

In einer Microservices-Architektur kommunizieren viele Services miteinander. Mit wachsender Anzahl von Services und Nutzer:innen steigt der Traffic exponentiell – ohne Load Balancing entstehen Engpässe und Single Points of Failure.



Einführung: CI/CD



Was ist CI/CD?

CI/CD steht für *Continuous Integration* und *Continuous Delivery/Deployment* – die Automatisierung des gesamten Software-Lieferprozesses.

Build

Code wird automatisch kompiliert und als Container-Image gebaut

Test

Automatisierte Tests prüfen die Qualität des Codes

Deployment

Das getestete Image wird automatisch in die Zielumgebung ausgeliefert

CI/CD Pipeline: Ablauf

1

1. Code Commit

Entwickler:in pusht Code ins Repository (z.B. Git)

2

2. Build Image

Die Pipeline baut automatisch ein neues Docker-Image

3

3. Tests

Unit-, Integrations- und ggf. End-to-End-Tests laufen automatisch

4

4. Deployment

Bei Erfolg wird das neue Image automatisch in Kubernetes ausgerollt

Durch eine CI/CD-Pipeline wird aus jedem Code-Commit ein potenzielles Release – schnell, reproduzierbar und mit minimalem manuellem Aufwand.

Rolling Update beobachten

Ziel: Ein Deployment aktualisieren und beobachten, wie Kubernetes Pods schrittweise ersetzt.

Arbeitsordner: uebung-14-rolling-update

Aufgabe

1. Wende zuerst `deployment-v1.yaml` an.
2. Prüfe die laufenden Pods.
3. Spiele danach `deployment-v2.yaml` ein.
4. Beobachte mit `kubectl rollout status`, wie das Update durchgeführt wird.
5. Prüfe, welches Image danach aktiv ist.

Hilfreiche Befehle

```
kubectl apply -f deployment-v1.yaml
kubectl get pods
kubectl apply -f deployment-v2.yaml
kubectl rollout status deployment/webapp
kubectl describe deployment webapp
```

Deployment-Strategien



Rolling Update

Pods werden schrittweise ersetzt. Während des Updates laufen immer alte und neue Pods gleichzeitig – **kein Downtime**, aber beide Versionen kurzzeitig aktiv.



Blue-Green Deployment

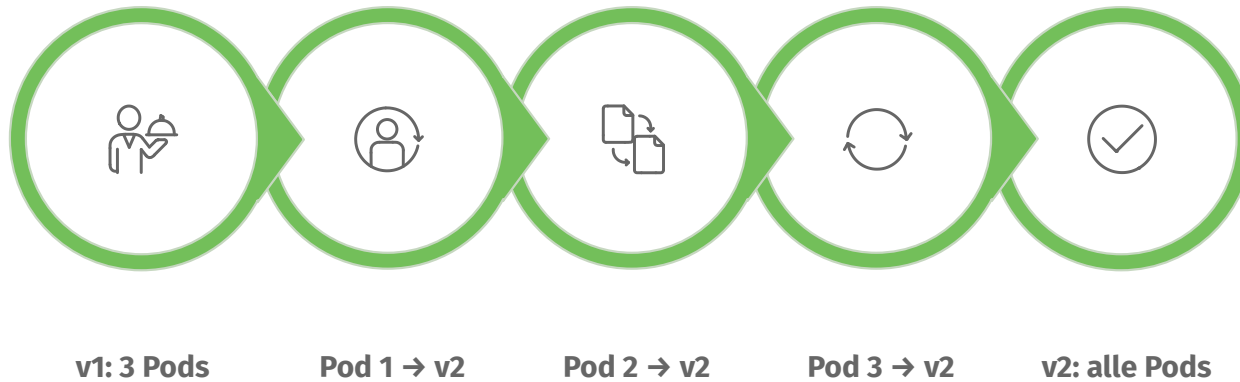
Zwei identische Umgebungen existieren parallel. Der Traffic wird vom alten System (Blue) auf das neue (Green) umgeschaltet – sofortiger Rollback möglich.



Canary Releases

Ein kleiner Teil des Traffics (z.B. 5%) wird auf die neue Version geleitet. Bei Erfolg wird schrittweise mehr Traffic umgeleitet – minimales Risiko.

Beispiel: Rolling Update



Kein Downtime beim Update

Beim Rolling Update ersetzt Kubernetes alte Pods **schrittweise** durch neue. Dabei ist immer mindestens eine Instanz verfügbar – der Service bleibt durchgehend erreichbar.

- ✓ Rolling Updates sind die **Standardstrategie** in Kubernetes – einfach konfigurierbar und für die meisten Anwendungsfälle geeignet.

Diskussion: Deployment-Strategien

Denkt über die vorgestellten Strategien nach und diskutiert gemeinsam:

Welche Strategie ist sicherer?

Welche Strategie minimiert das Risiko, dass Nutzer:innen eine fehlerhafte Version zu sehen bekommen? Was ermöglicht den schnellsten Rollback?

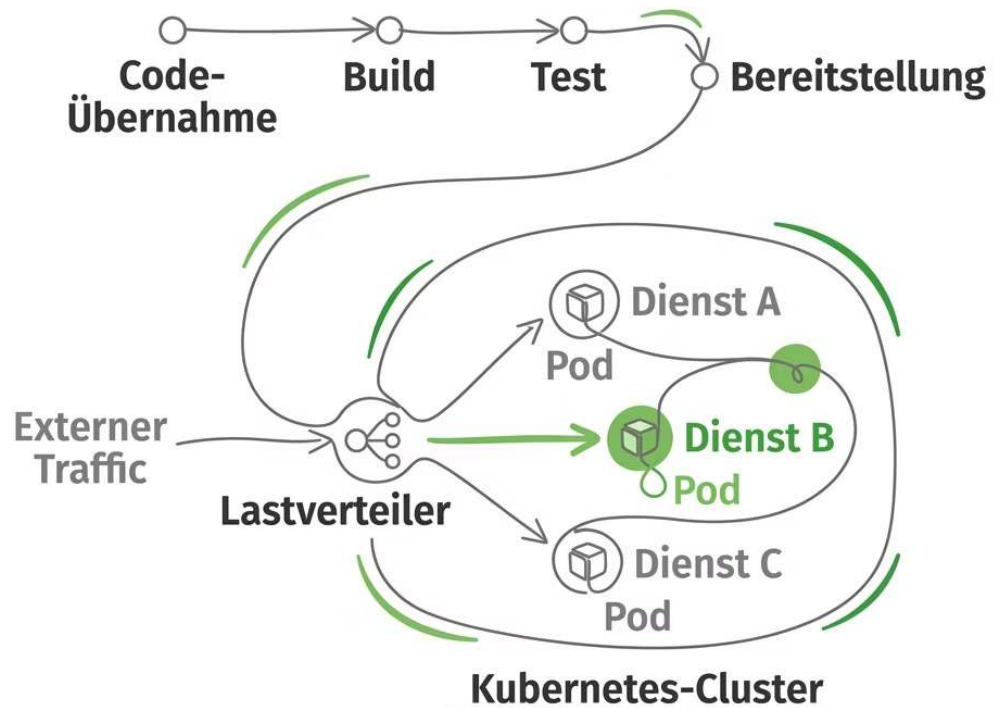
Welche Strategie ist schneller?

Welche Strategie bringt neue Features am schnellsten in Produktion? Wo liegen die Kompromisse?

  Schreibt eure Antworten in den Chat!



Gesamtarchitektur: Microservices mit Kubernetes



Alle Bausteine zusammen

In einer produktiven Microservices-Umgebung greifen alle Komponenten ineinander:

- **Mehrere Services**

Jeder Service hat seinen eigenen Deployment-Lifecycle

- **Load Balancer**

Verteilt externen Traffic auf die richtigen Services

- **Container & Kubernetes**

Jeder Service läuft in Pods, orchestriert von Kubernetes

- **CI/CD Pipeline**

Bringt Änderungen automatisiert und sicher in Produktion

Kleine Kubernetes-Architektur lesen

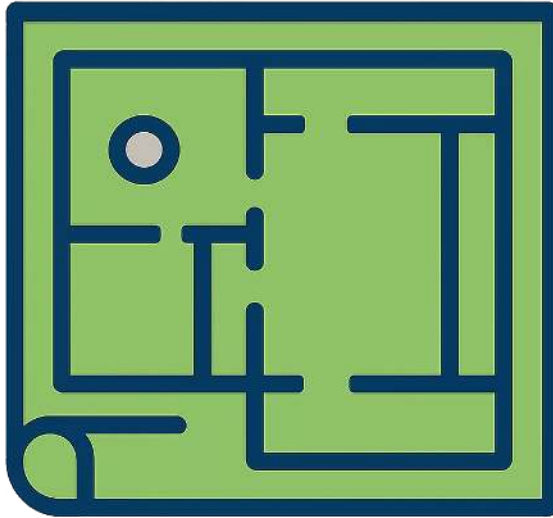
Ziel: Die Grundbausteine eines kleinen Kubernetes Setups gemeinsam einordnen.

Arbeitsordner: uebung-15-kubernetes-architektur

Aufgabe

1. Lies `deployment.yaml`, `service.yaml` und `configmap.yaml`.
2. Benenne, welche Rolle jede Ressource im Setup hat.
3. Erkläre, wie App, Konfiguration und Netzwerkzugriff zusammenspielen.
4. Überlegt in Kleingruppen, welche Ressource man als Nächstes ergänzen würde.

Transferaufgabe: Eigene Architektur entwerfen



Beispiel: Online-Shop

Entwerft in Stichpunkten eine Kubernetes-Architektur für einen Online-Shop oder eine eigene Idee. Denkt dabei an:

  Stichpunkte in den Chat – wir besprechen gemeinsam!

1

Welche Services?

z.B. Produkt-Service, Warenkorb, Bestellung, Authentifizierung

2

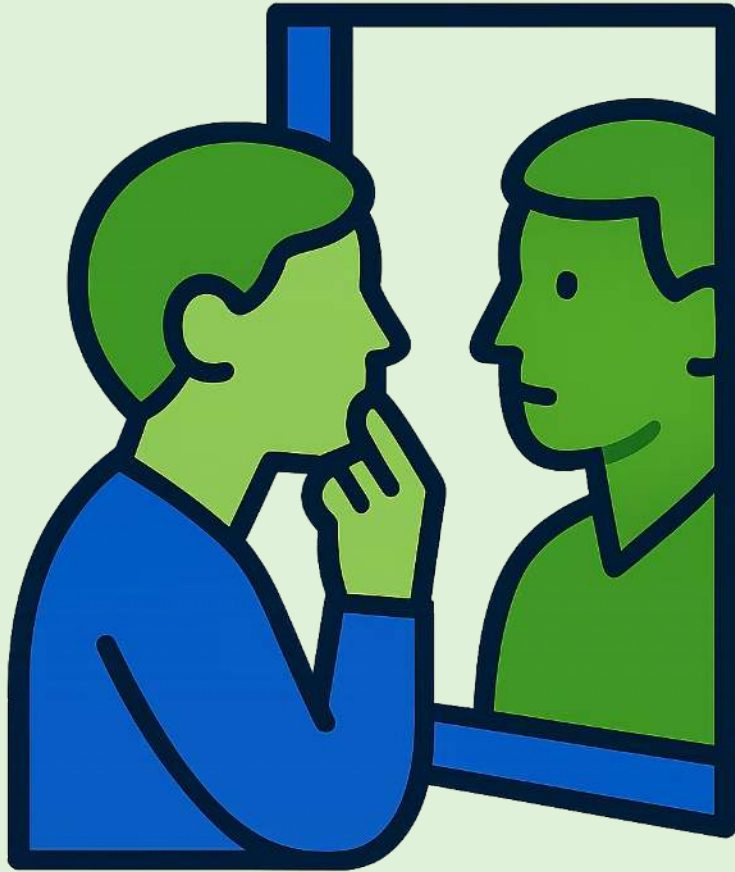
Wie skalieren?

Welche Services brauchen mehr Replikas? Wann und wie?

3

Wie deployen?

Welche Deployment-Strategie passt für welchen Service?



Reflexion

Was nehmt ihr mit?

Was war neu?

Welche Konzepte habt ihr heute zum ersten Mal gehört oder wirklich verstanden?

Was war schwierig?

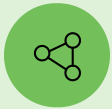
Wo gab es Verständnisfragen oder Unsicherheiten? Was braucht noch mehr Übung?

Wo seht ihr Einsatz?

In welchen eurer eigenen Projekte oder zukünftigen Jobs könnte Kubernetes sinnvoll sein?

Zusammenfassung

Das sind die vier Kernthemen dieser Einheit – bereit für die Praxis:



Kubernetes orchestriert Container

Deklarativer Desired-State-Ansatz – Kubernetes überwacht und korrigiert kontinuierlich



Skalierung & Self-Healing

Replicas für horizontale Skalierung, automatischer Neustart bei Fehlern



Load Balancing

Kubernetes Services verteilen Traffic stabil und transparent auf Pods



CI/CD für Deployment

Automatisierter Build-, Test- und Release-Prozess via Pipeline

Abschluss & Ausblick

Kursende

Ihr habt alle vier Unterrichtseinheiten abgeschlossen – von Microservices-Grundlagen bis hin zu Kubernetes und CI/CD.

Offene Fragen

Gibt es noch Unklarheiten zu Pods, Deployments, Services oder Deployment-Strategien? Jetzt ist der richtige Moment!

Weiterführende Themen

Helm Charts, StatefulSets, Persistent Volumes, Ingress Controller, Service Mesh (Istio), Monitoring mit Prometheus & Grafana

